



Day 3

Review, Security & Permissions

AI-written code ships only through gates the team trusts. Today the gates get built - on a running **Command Center** build, the same kind of system you may ship at LVT.

By Shivam Patel · [LinkedIn](#)

SET YOURSELF UP TO WIN

Learn These Success Hacks

- ✓ Attend every session
- ✓ Be attentive in the class
- ✓ Participate in class activities and demos
- ✓ Actively advocate for yourself
- ✓ Raise post-class help on the WhatsApp Group
- ✓ Ensure you have WhatsApp Group access
- ✓ Consistency is the key
- ✓ Be patient with yourself!



Don't worry!

We are here to help at your
every step!

SETUP · GET THE CODE

Clone the Repo

Get the Command Center build running on your machine - the same app we review, secure, and gate all day.

1

Clone the repo

git clone the URL, then cd into the project.

2

Install dependencies

make setup pulls backend + frontend (pnpm).

3

Run the tests

make test - everything green on the baseline.

```

● ● ● claude-security - zsh

# clone & enter
git clone \
  https://github.com/anshajk/claude-security.git
cd claude-security

# install + verify (Node 20+ · pnpm)
make setup
make test    ✓ all green

```

Repo github.com/anshajk/claude-security · branch **main** · Node 20+ & pnpm



Heads-up

This repo hides intentional vulnerabilities for the labs - never copy these patterns into production.

The day's one idea

An AI first draft that **looks** idiomatic is the most dangerous kind of code, because tidiness disarms the reviewer.

So we stop trusting how code reads, and start trusting what it passes:

- **Self-verifying loops** - the agent catches its own mistakes first
- **Independent review** - judgments that are not the author's
- **Secure-by-default tooling** - catch vulns while the agent writes
- **A permissions posture** - flow without footguns, committed as a team standard

Mindset: review effort is a budget. Spend it on what only a human can catch; move everything a machine can catch to a deterministic gate.

Agenda For Today

#1



Anti-slop & self-verify

- Catch slop with a checklist + wire a self-verify loop

#2



Multi-hat adversarial review

- A review panel that surfaces what one reviewer hides

#3



Security: OWASP & secure-by-default

- Project-scoped security setup + triage muscle

#4



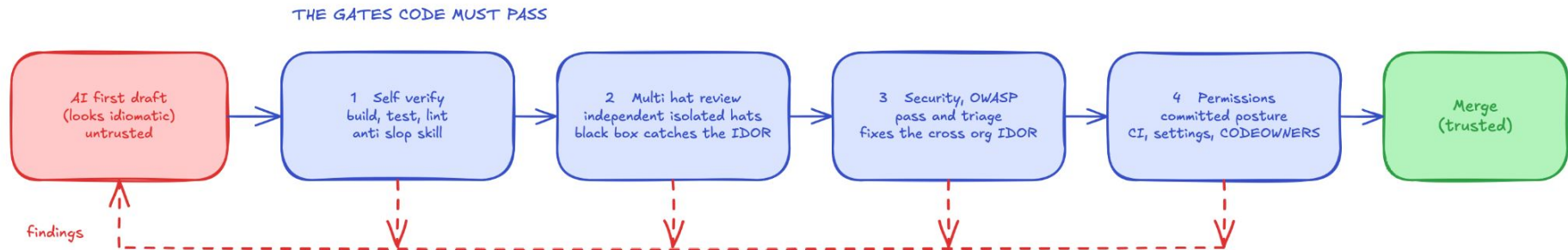
Permissions & enforcement

- Every build carries all four gates, or logged findings



Let's Get Started!

The Trust Gate



Cheap deterministic gates first, expensive human judgment last.
Findings loop back, the draft is untrusted until the gates are green.

Cheap deterministic gates first; expensive human judgment last. Findings loop back - the draft is untrusted until the gates are green.



Anti-Slop & Reviewing AI Code

Make review of AI code calibrated, not exhausting.

What slop looks like in practice

- **Plausible-but-wrong helpers** - an inverted comparison that reads perfectly
- **Copy-paste drift** - a block lifted from elsewhere, status/route/field not adapted
- **Dead branches** - conditions that can never be true given the data model
- **Tests that assert nothing** - green lights with no engine behind them
- **Off-by-one** - `rows[: limit - 1]` - “show 10” shows 9

Why veterans miss it: slop pattern-matches as idiomatic. You read the shape, nod, and move on. The shape is what the model gets right; the logic is what it gets wrong.

Spot the slop – how many bugs?

```
// backend/src/api/acks.ts – AI draft of "acknowledge alert"
// priority is 'high' | 'low'    status is 'open' | 'resolved'
export function isOverdue(priority: string, minutesSince: number): boolean {
  const sla = ACK_SLA[priority] ?? 60;
  return minutesSince < sla;
}

app.post("/alerts/:alertId/acknowledge", async (req) => {
  if (row.priority === "critical") {
    app.log.warn(...);
  }
  conn.prepare("UPDATE alerts SET status='resolved' WHERE id=?").run(alertId);
  try {
    conn.prepare("UPDATE alerts SET notes=? WHERE id=?").run(body.notes, alertId);
  } catch {}
});
```

Idiomatic, compiles, the test is green. Count the bugs silently - write down a number. Don't say it yet.

Slop clusters: One weak gate hides several bugs

```
export function isOverdue(priority: string, minutesSince: number): boolean {
  const sla = ACK_SLA[priority] ?? 60;
  return minutesSince < sla;          // (1) inverted: overdue when LESS time passed
}

app.post("/alerts/:alertId/acknowledge", async (req) => {
  if (row.priority === "critical") {   // (2) dead branch – only high|low exist
    app.log.warn(...);
  }
  conn.prepare("UPDATE alerts SET status='resolved' WHERE id=?").run(alertId);
  try {                                // (3) acknowledge silently RESOLVES (drift)
    conn.prepare("UPDATE alerts SET notes=? WHERE id=?").run(body.notes, alertId);
  } catch {}                          // (4) swallowed error – 200, no notes saved
});
```

Four defects in twelve idiomatic lines. A vacuous test green-lights all of them.

The fix: a system, not a harder stare

A scored checklist - read for intent in five passes:

- Correctness & logic (re-derive each helper)
- Copy-paste drift (right identifier, right object?)
- Error handling (swallowed failures?)
- Tests that assert nothing
- Hygiene (dead code, misleading comments)

A self-verifying loop - the agent runs build + test + lint and self-critiques **before** you see the diff:

```
/self-verify  
# 1 lint+type 2 tests 3 build  
# 4 structured self-critique  
# 5 present diff + findings
```

Scaffolding vs. gate: line-by-line reading is scaffolding that builds intuition; you do less as tooling grows. Tests, lint, and contracts are permanent gates.

Self-review: the agent as its own first reviewer

Before a human reads a line, the change should already have reviewed itself. Two kinds of review, doing two different jobs:

Scaffolding - fades over time

Reading the diff line-by-line builds your instinct for what slop looks like. Valuable now; you do less of it as the gates mature.

Human attention is the scarce resource; don't spend it on what a machine can check.

Permanent gate - always on

Tests, lint, type-check, contracts. Deterministic, tireless, and run every single time; no judgment call required.

These never fade: they are deterministic and predictable

The move: let the machine catch machine-catchable mistakes, so human judgment goes where it actually counts.

How /self-verify works - draft to presented diff

Treat your own first draft as untrusted. Three steps, in order:

1 · Deterministic gates

lint / type-check → tests → build. Anything red: fix the cause and re-run.
Never present over a failing gate.

2 · Structured self-critique

re-read the diff for logic · copy-paste drift · error-handling · weak tests · scope creep.

3 · Present

show the gate results, the self-critique, then the diff - flag anything you're unsure about instead of smoothing it over.

```
$ /self-verify
make lint  && \
make test  && \
make build

# then: logic · drift · error-
#       handling · tests · scope
```

A green test that asserts nothing is red. Strengthen the assertion and re-run - the gates catch machine mistakes for free; the human still owns intent.



Adversarial Review: Multi-Hat Thinking

Get judgments that are not the author's.

One reviewer wearing four hats does each one poorly



Three hats - one session each, no cross-talk

Security

/hat-security

The attacker's lens: injection, IDOR, SSRF, XSS, hard-coded secrets, and authz on state-changing or physical-effect actions.

Design / DRY

/hat-design

The six-months lens: copy-paste drift, wrong boundaries, misleading names, swallowed errors; what will hurt to maintain.

Black-box

/hat-blackbox <url>

Contract-only; may NOT read the source. Probes what the API promises and reports where reality breaks the promise.

Separate sessions = Independence.

One context blends the conflicts away; three vantages stay sharp and are allowed to disagree. The design and security hats will clash on the ORDER BY, and that clash is the point.

Run the panel, then compare the verdicts

The value isn't any one hat - it's the deltas between them.

1. Capture each hat verbatim - run them separately so one hat's output can't prime the next.
2. Line the panel up against a single all-in-one reviewer.
3. Read the deltas: what did an independent hat catch that the blended pass smoothed over?

What the comparison surfaces

Coverage gap - the black-box hat makes the cross-tenant request; the tidy-code reviewer never does.

Surfaced tradeoff - where the design and security hats disagree, the decision is now explicit instead of hidden.

Same object, a new lens: the clips IDOR you black-box tested returns in Part 3's code review - one thread seen from three angles.

The black-box hat can't be talked out of a bug

It sees **only** contracts/openapi.yaml - never the implementation. So it:

- probes what the contract promises
- can't anchor to the code's intent
- can't quietly weaken a test to pass

The contract promises every resource is org-scoped. So the hat tries the obvious attack:

```
# Acme token, Globex's clip.  
# Contract promises 404.  
curl /clips/clip_globex_1 \  
  -H "Authorization: Bearer acme-..."  
  
# Actual: 200 + Globex footage URL  
# → CONTRACT VIOLATION (IDOR)
```

Caught from the contract alone. The all-in-one reviewer reading tidy code may never make this request.

Keep a human involved

Never ship AI code with **only** AI-written tests.

Keep at least one **human-authored** assertion on anything that matters.

```
// backend/tests/tenant_isolation.test.ts (HUMAN-authored)
test("cross-org clip access is denied", async () => {
  const res = await client().inject({ url: "/clips/clip_globex_1",
    headers: { authorization: "Bearer acme-operator-token" } });
  expect(res.statusCode).toBe(404); // cross-tenant footage leak (IDOR)
});
```

LIVE DEMO

Live Demo: Part 2



```
claude - demo session
```

In this demo, we will:

- Open two separate sessions - /hat-security and /hat-blackbox - independence is the point.
- Capture each hat's findings verbatim; don't let them see each other's output.
- Compare against one all-in-one reviewer: what did an independent hat catch that it smoothed over?
- Each engineer writes one human-authored assertion on their critical path.

Outcome: engineers get judgments that aren't the author's - and ship at least one human assertion an agent's happy-path tests would never write.

PART 2.1

Github Actions

GITHUB ACTIONS: Two ways to trigger the action

anthropics/claude-code-action@v1 runs Claude Code as a workflow step. Your code stays on GitHub's runners.

1 Interactive

```
@claude fix the TypeError in the dashboard
```

Fires on @claude in an issue or PR comment

Needs no prompt input; the phrase is the trigger

2 Automation

```
prompt: "/owasp-review backend/src/api"
```

Fires on pull_request, schedule, workflow_dispatch

Needs prompt: set; that is what picks this mode

Beta to v1: what moved

BETA	V1
mode:	gone, auto detected
direct_prompt	prompt
custom_instructions	claude_args: --append-system-prompt
max_turns	claude_args: --max-turns
model	claude_args: --model
allowed_tools	claude_args: --allowedTools
@beta	@v1

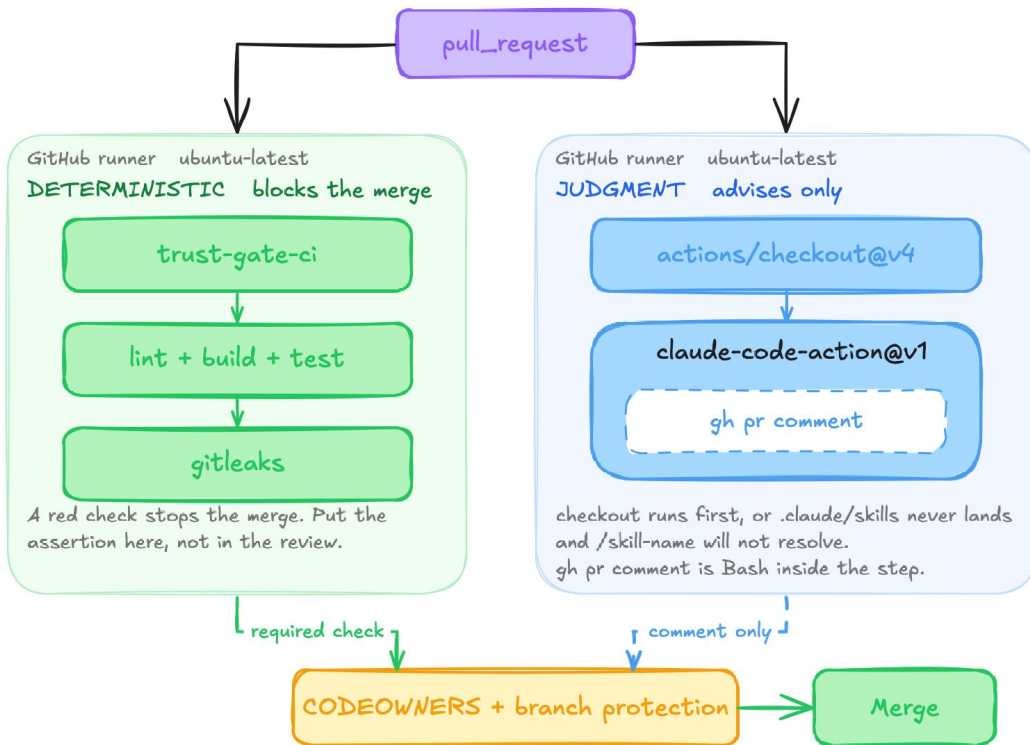
@v1 is a moving tag. A gate you trust pins to a commit SHA.

Setup

/install-github-app installs the GitHub App and wires the ANTHROPIC_API_KEY secret · repo admin only · App scopes: Contents, Issues, Pull requests (read and write)

GITHUB ACTIONS: ARCHITECTURE

How a PR moves through the gates : Separate jobs mean separate runners. Only the deterministic lane can stop a merge.



GITHUB ACTIONS: THE WORKFLOW FILE

Every line that is not boilerplate is a decision about scope, cost, or trust.

```

part3-security.yml

on:
  pull_request:
    paths: ["backend/src/**", "contracts/openapi.yaml"]

concurrency:
  group: part3-security-${{ github.event.pull_request.number }}
  cancel-in-progress: true

permissions:
  contents: read
  pull-requests: write

jobs:
  owasp-pass:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${ secrets.KEY }
          github_token: ${ secrets.GITHUB_TOKEN }
          prompt: |
            /owasp-review backend/src/api
          claude_args: |
            --permission-mode dontAsk
            --max-turns 20 --model sonnet
            --allowedTools "Read,Grep,Bash(gh pr comment:*)"
  
```

- 1 **paths:**
Wakes only on the vulnerable surface. Advisory jobs only.
- 2 **concurrency**
A force push cancels the superseded run.
- 3 **permissions:**
Scopes GITHUB_TOKEN. Binds Claude only if you pass it in.
- 4 **timeout-minutes**
Caps the job. --max-turns caps the loop.
- 5 **checkout first**
Before the action, or .claude/skills never lands.
- 6 **github_token**
Skips OIDC. Makes permissions: the real boundary.
- 7 **prompt**
Plain text or a skill. Flags live in claude_args.

Two coherent auth choices. Pick one.

github_token: \${ secrets.GITHUB_TOKEN }
Skips the OIDC exchange, scoped to the job's declared permissions, expires with the job. Comments post as github-actions[bot].

default app auth + id-token: write
OIDC token is required for the Claude app. Comments post as claude[bot], but permissions: no longer bounds Claude.

GITHUB ACTIONS: PERMISSIONS

Permission boundaries and the CI trap :Two independent boundaries. Getting one right does not cover the other.

1. Tool boundary

```
--allowedTools + --permission-mode dontAsk
```

Listed tools are approved. Anything else is denied outright instead of prompting.

Bash(gh pr comment:*)

The real allowlist. Bash is disabled by default.

Read(./env) in deny, not allow

Read-only tools need no approval inside the working directory, so an allow rule scopes nothing.

ls cat grep find

A built-in read-only set that runs without a prompt in every mode.

2. Token boundary

```
permissions:
```

Scopes secrets.GITHUB_TOKEN. It only reaches Claude if you pass github_token:.

you pass github_token:

permissions: is the real boundary, and the token expires with the job.

you do not

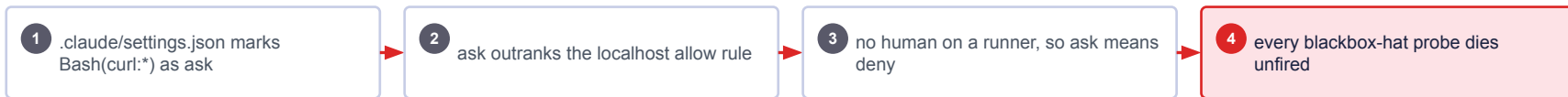
The action exchanges an OIDC token at api.anthropic.com for an app token requesting contents: write, pull_requests: write, issues: write.

Bash(curl http://localhost/ *)

Argument patterns are fragile: this misses curl -X GET, https://, -L redirects, and variables.

The headless trap: in CI, ask means deny

deny, then ask, then allow. First match wins; specificity does not reorder.



Fix delete the ask rule (default mode already prompts locally), allow the probe in claude_args, and run **--permission-mode dontAsk**

Not the fix **bypassPermissions** skips prompts except those forced by an explicit ask rule, so the trap survives it, and it approves every tool rather than only the ones you listed.

GITHUB ACTIONS: THE MERGE GATE

Which gates block, which only advise : Claude jobs post findings and exit clean. Severity never fails a build, by design.

Blocks

trust-gate-ci

- biome lint and format check
- pnpm -r build (type check)
- pnpm -r test (vitest)
- gitleaks secret scan

Any red check stops the merge.

Advises

claude-code-action@v1

- security-hat ranked findings
- blackbox-hat contract vs behavior
- owasp-pass structured report
- security-reviewer second witness

Each posts exactly one PR comment.

Want a finding to block? Write the assertion.

A review comment is a claim. A failing test is a proof. Move the cross tenant clips test into the backend suite and trust-gate-ci stays red until the IDOR is fixed.

CODEOWNERS holds the paths where an AI diff is most expensive to get wrong:

`/backend/src/core/`

`/.claude/`

`/.github/`

Cost controls

<code>paths:</code>	advisory jobs only
<code>cancel-in-progress</code>	kills stale runs
<code>--max-turns</code>	caps the loop
<code>timeout-minutes</code>	caps the job

Never path-filter a required check: the PR waits on a run that never happens and stays pending forever.



Security: Secure-by-Default & OWASP

Catch issues while the agent writes and keep the tooling honest.

The vulnerability classes you'll meet today

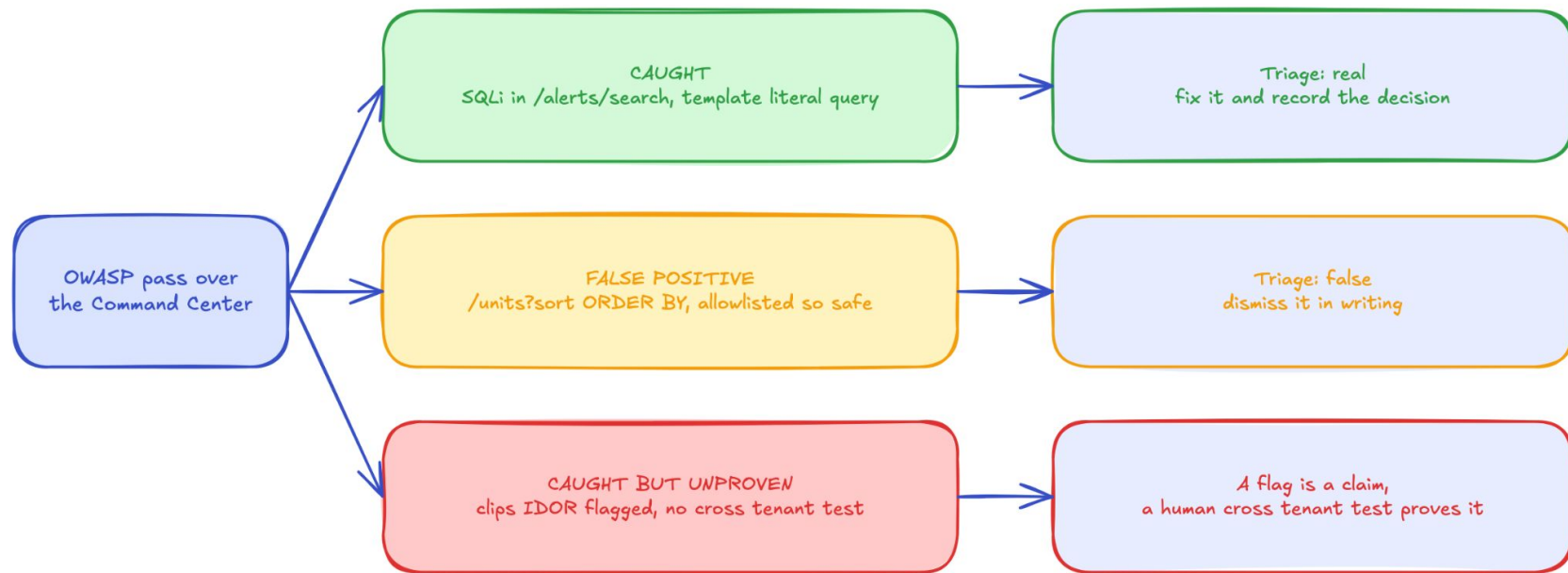
OWASP names the most common ways web apps get hacked, so we can all point at the same bug. Five classes show up in today's Command Center build:

Class	In plain English
Injection (SQLi)	User text glued into a query runs as code, not data
Broken access control / IDOR	A missing tenant filter lets one user read another's data
SSRF	Server fetches a user-supplied URL → reaches internal systems
Stored XSS	User input rendered as HTML/JS in someone else's browser
Secrets / misconfig	Hard-coded keys, or wide-open settings (CORS *, debug on)

One idea to hold onto: authenticated ≠ authorized. Logging in proves who you are; it does not prove you're allowed to see this row.

Tooling output is a starting point, not a verdict

Three buckets every security pass produces. Teaching all three is the point - blind trust in the tool is just a flavor of over-trust.



The same pass also flags SSRF and stored XSS, five real issues plus one decoy. These three are the teaching buckets. A tool flag is a claim, not a proof.

Secure-by-default: a project-scoped skill

Installed at `/.claude/skills/owasp-review/` so every engineer inherits identical checks. The agent applies secure patterns as it writes and explains flagged risks - real-time coaching.

Marketplace = attack surface. Skills & plugins run with the agent's privileges. Prefer reputable sources, read the skill before installing, pin versions, treat adoption as its own security review.

```
# .claude/skills/owasp-review/SKILL.md
---
name: owasp-review
description: >
  Reviews for OWASP Top 10 – injection,
  broken access control/IDOR, SSRF, XSS,
  secrets. Use for any security pass.
---
# explicit anti-false-positive rule:
# allowlist-guarded ORDER BY is
# reviewed-safe, NOT a vulnerability
```

OWASP also ranks the skills themselves: AST10

The web Top 10 is about bugs in **your** app. The Agentic Skills Top 10 (AST10) is about the risk of the skills and plugins you install into your agent - a skill runs with **your agent's privileges**.

Mental model: MCP is how the model talks to tools; AST10 is about what those tools actually do. Installing a skill is running someone else's code inside your agent.

The “lethal trifecta” - a skill is dangerous when it has all three:

Condition	What that looks like
1 · Reads private data	SSH keys, API tokens, .env, browser data
2 · Sees untrusted content	skill text, memory files, a web page, an email
3 · Can talk to the outside	network egress, webhooks, curl

Not theoretical (2026): Snyk's ToxicSkills audit scanned 3,984 skills - 36.8% had security flaws, 13.4% critical, with 76+ confirmed malicious payloads; researchers since count 135,000+ exposed instances.

THE LIST

The Agentic Skills Top 10

#	Risk	What it means
AST01	Malicious skills	The skill is malware - it does harm on purpose.
AST02	Supply-chain compromise	A trusted skill's source or registry gets poisoned.
AST03	Over-privileged skills	Asks for far more access than it needs.
AST04	Insecure metadata	The manifest itself hides a payload or lies about what it does.
AST05	Untrusted external instructions	Skill pulls in text (files, web, email) that hijacks the agent - prompt injection.
AST06	Weak isolation	Runs on your host, not sandboxed - one bug means full-machine access.
AST07	Update drift	A safe version silently updates into a malicious one.
AST08	Poor scanning	Pattern-matching scanners miss natural-language instruction attacks.
AST09	No governance	Nobody tracks which skills are installed or what they can reach.
AST10	Cross-platform reuse	Porting a skill between tools drops its safety metadata.

Treat every skill like code you're about to run

- **Read it before you install it:** It inherits your agent's permissions
- **Prefer reputable sources, pin versions:** Treat adopting a plugin as its own security review
- **Least privilege + a strong deny list** (Part 4): Is what contains a bad skill
- **Governance:** Know which skills are installed and what they can reach

That's exactly why this bootcamp ships its **own vetted, self-contained owasp-review** skill; no marketplace install, nothing pulled from the network.

Part 3's marketplace-as-attack-surface aside and Part 4's deny list are these AST10 mitigations put into practice.

Closest to home: Claude Code CVE-2025-59536 (CVSS 8.7) & CVE-2026-21852 - opening an untrusted repo could run shell or leak API keys before any trust prompt. Repo config is part of the execution layer - which is why Part 4 gates .claude/ edits.

Quick test: if a skill can read secrets, read untrusted input, and reach the network, scrutinise it hard - that's the lethal trifecta.

A safer skill: least privilege, signed, pinned

OWASP proposes a universal manifest so these become declarable and checkable, not trust-me prose:

```
permissions:
  files:
    read: [ ~/.config/app.json ] # explicit, no wildcards
    deny_write: # never auto-write memory
      [ SOUL.md, MEMORY.md, AGENTS.md ]
  network:
    allow: [ api.example.com ] # allowlist, not on/off
    deny: "*" # default-deny egress
    shell: false # declared, not implicit
  signature: ed25519:... # provenance
  content_hash: sha256:... # tamper-evident
  risk_tier: L1 # L0 safe ... L3 destructive
```

- **Least privilege:** Explicit paths, minimal scope (AST03)
- **Default-deny egress:** Allowlist domains, not network: true
- **Protect identity files:** No auto-writes to memory (AST03/05)
- **Signed + hashed + pinned:** Immutable, no version ranges (AST01/02/07)
- **Risk tier:** Enables automated governance (AST09/10)

The instinct is familiar: it's npm/pip hygiene applied to skills. Minimal permissions, pinned and signed, and never let a skill silently write the agent's own memory.

How the owasp-review skill actually reviews

A fixed method. Five categories, checked in order:

1. **Injection** - SQL / command / template built from request data.
2. Broken access control / **IDOR** - compare sibling handlers for a tenant predicate.
3. **SSRF** - server-side fetch of a client-supplied URL with no allowlist.
4. **XSS** - untrusted data reaching an HTML sink.
5. **Secrets & config** - hard-coded creds, permissive CORS, leaked URLs.

```
[SEVERITY: high] <class> – file:line
What:    the flaw, one line
Exploit: concrete attacker action
Fix:     minimal remediation
Confidence: high | med | low
```

The reviewed-safe rule: an allowlist-guarded ORDER BY is called safe, not flagged. Refusing a false alarm - in writing - is part of the method, not an afterthought.

An independent security-reviewer subagent + a deterministic hook

```
# .claude/agents/security-reviewer.md
---
name: security-reviewer
description: Independent OWASP pass on a diff
tools: Read, Grep, Glob, Bash(git diff:*)
model: inherit
---
# told to COMPARE sibling handlers -
# the clips IDOR is an inconsistency:
# units & alerts scope by org, clips doesn't
```

```
.claude/hooks/secret-scan.sh

# PreToolUse on Write|Edit → exit 2 = block
# greps proposed content for AWS keys,
# private keys, GitHub/Slack tokens...
# A deterministic gate the agent can't skip.
```

Two gate types: the subagent is judgment; the hook is deterministic. You need both.

From a finding to a written verdict - /triage

A flagged finding is a claim. Triage turns it into a verdict the team can stand behind:

1. Locate - quote the exact file:line.
2. Trace input → sink - reachable from an untrusted caller? For authz, is there a tenant predicate?
3. Verdict - real, or false-positive.
4. Record - in writing, in the triage template. Both a fix AND a dismissal.

```
$ /triage clips IDOR
# locate clips.ts:10
# trace getPrincipal() read, but
#       SELECT has no org_id filter
# verdict REAL - cross-tenant read
# record triage-record-template.md
```

No written justification? a false positive dismissed without a written reason is indistinguishable from a real bug ignored.

LIVE DEMO

Live Demo: Part 3

claude - demo session

In this demo, we will:

- Run the OWASP skill over backend/src/api/ and the alert-notes rendering.
- Run the security-reviewer subagent on multi-tenant scoping - independent confirmation of the IDOR, and it names the missing cross-tenant test.
- Triage one finding end-to-end - real or false-positive, written down either way.
- Watch the facilitator's secret-scan hook demo block a planted secret.

Outcome: You see the full gate - tool → independent review → written triage → deterministic hook

Where they were hiding - the answer key

Vulnerability	Where	The exploit, in one line	What it really is
SQL injection	alerts.ts · /alerts/search	q is spliced straight into the SQL string	Real & exploitable
Multi-tenant IDOR	clips.ts · /clips/:id	No org_id filter → read another org's footage	Real - no test proves it
False positive	units.ts · /units?sort=	ORDER BY is allowlist-guarded → actually safe	Actually safe
SSRF	units.ts · /units/:id/callback	Server fetches a unit-supplied URL (e.g. cloud metadata)	Real - hits internal svcs
Stored XSS	AlertNotes.tsx	dangerouslySetInnerHTML on operator notes	Real - runs in browser

The tool's list is not the answer key: whether the review flags each one varies run to run - that's the point. Trust the code you read and verified, not the tool's output.

PART 4

Permissions Without Constant Interruptions

Balance flow against safety and commit it as a team standard.

How Claude Code decides yes or no

Every time the agent wants to run a command, read a file, or fetch a URL, Claude Code checks your rules first. They live in three lists in `.claude/settings.json`:

deny

never allowed

ask

prompt me every time

allow

run without asking

It checks them in that order - deny, then ask, then allow - and the first match wins. How specific a rule is doesn't matter; the order does.

A broad deny beats a narrow allow. Deny `Bash(aws *)` and allow `Bash(aws s3 ls)`? Still blocked - a deny rule can't carry exceptions.

Rules are enforced by Claude Code, not the AI. "Don't do X" in a prompt or `CLAUDE.md` is a request, not a fence. Only `settings.json` rules and hooks stop an action.

See every active rule with `/permissions`. Check the file is valid with `claude doctor`.

Sort every action into allow, ask, or deny

allow

Safe, frequent, easy to undo. Don't break flow.

`reads · tests · lint · make · curl localhost`

ask

Powerful or reaches the outside world. Prompt every time.

`git push · WebFetch · edits to .claude/`

deny

Never. Off-limits no matter what.

`.env reads · 169.254.169.254 · rm -rf · force-push`

The physical-action rule: a real Command Center never auto-fires a deterrent (lights, strobe, audio on a Unit in the field). Same for the agent: never auto-allow anything irreversible or with a real-world effect.

You can also set a default mode: plan (look, don't touch) · default (ask on first use) · acceptEdits (auto-accept file edits) · auto (a classifier decides - research preview) · bypassPermissions (skip prompts - throwaway sandbox only). More prompts = safer; fewer = faster.

A rule that looks safe can still be wide open

1. "Only curl this one site"

```
// ask: Bash(curl example.com *)
```

Doesn't stop any of these:

```
curl -X GET example.com      # flag first
curl https://evil.example.com # other host
curl bit.ly/xyz              # redirects
U=evil.com && curl $U         # variable
```

Fix: deny curl/wget, allow only WebFetch(domain:example.com).

2. "Let the agent use npx"

```
// allow: Bash(npx *)
```

Runners like npx, docker exec, and devbox run execute whatever you hand them - so this also allows npx rimraf . (delete via a helper) or npx any-package (run unvetted code). Claude Code doesn't unwrap them.

Fix: name the exact inner command - Bash(npx prettier *) - one rule per tool.

A permission rule is a claim, just like a tool's finding. Test it before you trust it - that's the lab. The safe default is allow narrow, deny broad.

Where rules live - and who can override them

Scope	Where	Who it affects
User	~/.claude/	Just you, in every project
Project	.claude/settings.json (committed)	Everyone on this repo
Local	.claude/settings.local.json (gitignored)	Just you, this repo only
Managed	Pushed by IT / DevOps	The whole company - can't be overridden

When they disagree: Managed > Local > Project > User. But the lists merge - every level's deny rules all apply together.

The fleet move (this is LVT's world): IT ships one Managed policy so no engineer can switch off the deny list or turn on "skip all prompts" - disableBypassPermissionsMode, disableAutoMode, allowManagedPermissionRulesOnly. It's also how you contain a risky skill or MCP server: deny mcp__*, block plugin marketplaces.

One rule has a blind spot - so layer them

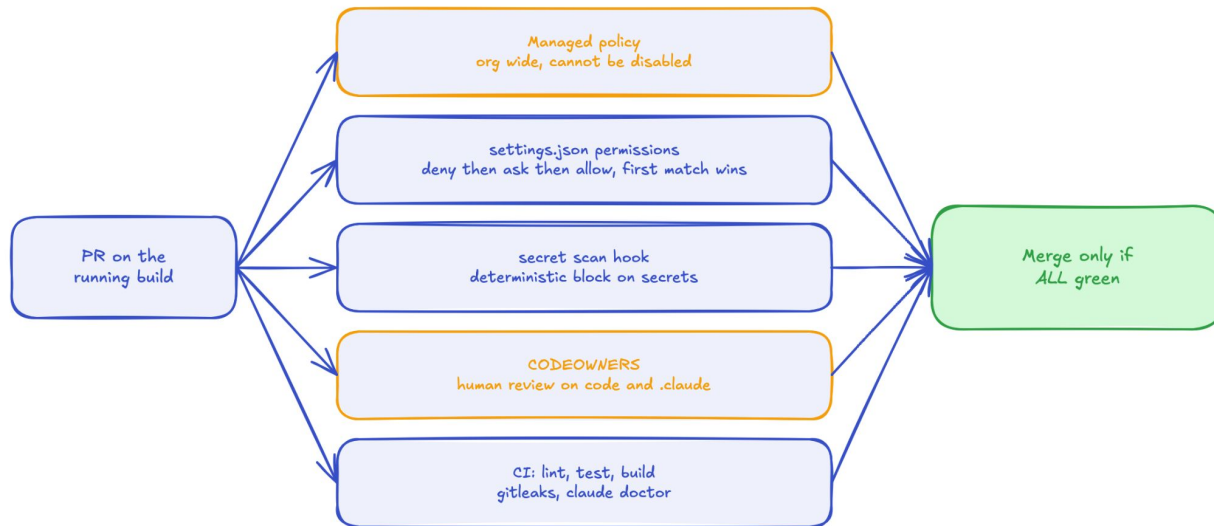
The blind spot: deny Read(.env) stops the agent's own file-read tool and commands it recognises like cat .env. It does not stop a script the agent writes - python read_env.py opens .env itself and walks right past the rule.

For anything that must never leak (the .env, a footage URL, the deterrent), protect it with three layers:

Layer	What it is	Catches
1 · Permission rule	deny Read(.env) in settings.json	The obvious path - agent tools + known commands
2 · PreToolUse hook	Your own script (like Part 3's secret-scan)	Anything you can detect - even past an allow rule
3 · Sandbox	The OS fences off files & network	Even a script the agent writes - survives injection

Defense in depth: each layer catches what the one before it misses. No single gate is enough on its own.

The full gate: every layer green, or no merge



Gate the gate: put CODEOWNERS on .claude and .github too, so changing the rules themselves (permissions, hooks, skills, CI) needs a human review.

Every layer is enforced by tooling, not the AI, and no single gate is enough.
The two gold layers, managed policy and CODEOWNERS, are the ones the agent cannot turn off.

Gate the gate: put CODEOWNERS on .claude/** too, so changing the rules themselves needs a human review.

LIVE DEMO

Live Demo: Part 4

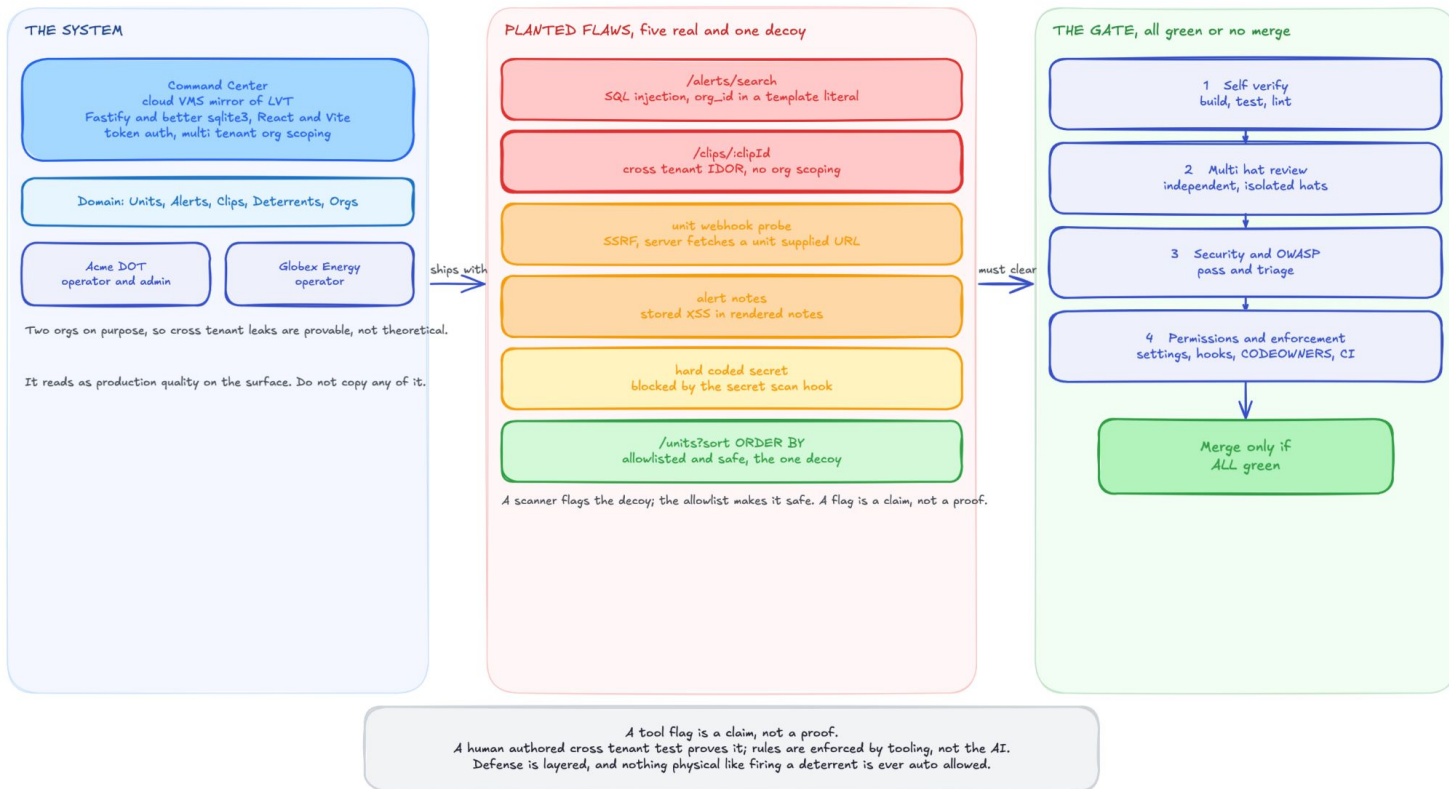
claude - demo session

In this demo, we will:

- Read your rules. Run `/permissions` to see them all; walk `settings.json` - for each: allow / ask / deny, and why? Then `claude doctor` to check it's valid.
- Open `fragile-rules.md` - 3 rules that look safe.
- Layer a gate. Follow `layered-gate.md`: protect `.env` with a deny rule, watch a script slip past it. Discuss what the sandbox would add.
- Decide & commit. Debate `contested-rule.md` (auto-allow git push to feature branches?). Record the decision and the dissent, then commit `settings.json`.

Outcome: a committed permissions baseline the whole team inherits - and the habit of testing a rule instead of trusting its name.

Recap



Repo map & commands

```
# one-time
make setup          # pnpm install (backend + frontend)
make test           # backend + frontend vitest (green on baseline)

# run the app (two terminals)
make dev-api        # Fastify Command Center → :8000 (/health)
make dev-web        # Vite dashboard → :5173

# Part 1
make slop           # apply planted defects to branch slop/part-1
make reset          # back to clean baseline

# artifacts
.claude/skills/      # self-verify, triage, hat-security|blackbox (manual /name),
                    # owasp-review (auto)
.claude/agents/security-reviewer.md
.claude/hooks/secret-scan.sh .claude/settings.json
labs/part{1..4}-*/   # README + checklist/rubric + answer keys
```

VERIFY before the session: hooks event names & schema · skills layout · Auto-mode naming · subagent frontmatter · .mcp.json shape - all carry VERIFY: notes in-repo.



CHECK YOUR UNDERSTANDING

Quiz Time!

Seven quick questions to lock in what we covered today.

QUIZ

 CHECK

Why is an AI first draft that looks idiomatic the most dangerous kind of code?

A

It is written faster than a human would write it

B

Idiomatic code cannot be unit-tested

C

Tidiness disarms the reviewer - you read the shape, nod, and move on

D

The compiler skips its checks on idiomatic code

Why is an AI first draft that looks idiomatic the most dangerous kind of code?

A

It is written faster than a human would write it

B

Idiomatic code cannot be unit-tested

C

Tidiness disarms the reviewer - you read the shape, nod, and move on ✓

D

The compiler skips its checks on idiomatic code

QUIZ

 CHECK

In /self-verify, what do you do with a green test that asserts nothing?

A

Treat it as red - strengthen the assertion and re-run

B

Accept it; a passing test is a passing test

C

Delete it so the suite stays green

D

Leave it for the security hat to handle

In /self-verify, what do you do with a green test that asserts nothing?

A

Treat it as red - strengthen the assertion and re-run



B

Accept it; a passing test is a passing test

C

Delete it so the suite stays green

D

Leave it for the security hat to handle

QUIZ

 CHECK

Why run three separate single-hat reviewers instead of one wearing all four hats?

A Running them in parallel is faster

B Each hat needs a different LLM

C A single hat can only check formatting

D One agent blends the conflicts away; independent hats go deep and can disagree

Why run three separate single-hat reviewers instead of one wearing all four hats?

A Running them in parallel is faster

B Each hat needs a different LLM

C A single hat can only check formatting

D One agent blends the conflicts away; independent hats go deep and can disagree ✓

QUIZ

? CHECK

What lets the black-box hat catch an IDOR the all-in-one reviewer misses?

A

It reviews the implementation line-by-line

B

It sees only the contract, not the code
- so tidy code can't talk it out of the bug

C

It runs with elevated database privileges

D

It rewrites weak tests until they pass

What lets the black-box hat catch an IDOR the all-in-one reviewer misses?

A

It reviews the implementation line-by-line

B

It sees only the contract, not the code - so tidy code can't talk it out of the bug ✓

C

It runs with elevated database privileges

D

It rewrites weak tests until they pass

QUIZ

? CHECK

What does “authenticated ≠ authorized” mean?

A

You must authenticate twice for risky actions

B

Authorization always happens before authentication

C

Logging in proves who you are, not that you may see this row

D

Only administrators are ever authorized

What does “authenticated ≠ authorized” mean?

A

You must authenticate twice for risky actions

B

Authorization always happens before authentication

C

Logging in proves who you are, not that you may see this row



D

Only administrators are ever authorized

QUIZ

? CHECK

A skill hits the “lethal trifecta” when it can do all three of what?

A

Read private data, see untrusted content, and reach the outside

B

Read code, write code, and run tests

C

Install plugins, auto-update, and log errors

D

Use the network, the disk, and the GPU

A skill hits the “lethal trifecta” when it can do all three of what?

A

Read private data, see untrusted content, and reach the outside ✓

B

Read code, write code, and run tests

C

Install plugins, auto-update, and log errors

D

Use the network, the disk, and the GPU

QUIZ

 CHECK

In what order does Claude Code check permission rules, and which wins?

A

allow → ask → deny; the most specific rule wins

B

deny → ask → allow; the first match wins - a broad deny beats a narrow allow

C

ask → allow → deny; the last match wins

D

There is no order; the AI decides from CLAUDE.md

In what order does Claude Code check permission rules, and which wins?

A

allow → ask → deny; the most specific rule wins

B

deny → ask → allow; the first match wins - a broad deny beats a narrow allow ✓

C

ask → allow → deny; the last match wins

D

There is no order; the AI decides from CLAUDE.md



Thank You!

See you on Day 4 🙌

AI Assisted Development
Program · Day 3